Latest updates: https://dl.acm.org/doi/10.1145/3703595.3705883

RESEARCH-ARTICLE

# Formalized Burrows-Wheeler Transform

**LOUIS CHEUNG**, University of Melbourne, Melbourne, VIC, Australia

**ALISTAIR MOFFAT**, University of Melbourne, Melbourne, VIC, Australia

**CHRISTINE RIZKALLAH**, University of Melbourne, Melbourne, VIC, Australia

**Open Access Support** provided by:

**University of Melbourne**

**PDF Download**
**3703595.3705883.pdf**
**20 January 2026**
**Total Citations:** 1
**Total Downloads:** 410

**Published:** 10 January 2025

**Citation in BibTeX format**

CPP '25: 14th ACM SIGPLAN International Conference on Certified Programs and Proofs
*January 20 - 21, 2025*
*CO, Denver, USA*

**Conference Sponsors:**
SIGPLAN

CPP '25: Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs (January 2025)
https://doi.org/10.1145/3703595.3705883
ISBN: 9798400713477

# Formalized Burrows-Wheeler Transform

Louis Cheung
The University of Melbourne
Melbourne, Australia
lfcheung@student.unimelb.edu.au

Alistair Moffat
The University of Melbourne
Melbourne, Australia
ammoffat@unimelb.edu.au

Christine Rizkallah
The University of Melbourne
Melbourne, Australia
christine.rizkallah@unimelb.edu.au

## Abstract

The Burrows-Wheeler transform (BWT) is an invertible lossless transformation that permutes input sequences into alternate sequences of the same length that frequently contain long localized regions that involve clusters consisting of just a few distinct symbols, and sometimes also include long runs of same-symbol repetitions. As a consequence, the BWT is used in a wide range of tools, from everyday file compression software, such as `bzip2`, to highly sophisticated bioinformatics tools, such as DNA sequencers, where an efficient encoding of genomic sequences is essential. As such, ensuring the correctness of the BWT is critical.

In this paper, we present the first formal verification of both the BWT and its inverse. Our verification efforts are conducted in Isabelle/HOL. Building on a recent formalization of the suffix array construction algorithm, we provide a mechanized proof of correctness, invertibility, and termination for both transformations. By doing so, we address the gap in the formal verification of compression algorithms, ensuring that the BWT can be safely deployed in critical applications such as genomic data analysis. This work thereby also provides the necessary foundation for verifying the various algorithms for compression and text search that operate on BWT-transformed sequences.

***CCS Concepts:*** • **Theory of computation → Logic and verification**; *Data compression*; *Pattern matching*; Sorting and searching; Program specifications; Program verification.

***Keywords:*** Formal methods, interactive theorem proving, Isabelle/HOL, formal verification, data compression, algorithmic verification, Burrows-Wheeler transform

**ACM Reference Format:**
Louis Cheung, Alistair Moffat, and Christine Rizkallah. 2025. Formalized Burrows-Wheeler Transform. In *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP '25), January 20–21, 2025, Denver, CO, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3703595.3705883

This work is licensed under a Creative Commons Attribution 4.0 International License.
*CPP '25, January 20–21, 2025, Denver, CO, USA*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1347-7/25/01
https://doi.org/10.1145/3703595.3705883

## 1 Introduction

The Burrows-Wheeler transform (BWT) [6] has received considerable attention since its conception in 1994, starting with initial disbelief that it could possibly make sense; and now, thirty years later, having progressed to widespread adoption in the fields of data compression and text search.

The importance of the BWT in data compression is due to two key features: localized subsequences and invertibility. First, the BWT transforms typical text inputs into globally non-homogeneous sequences that are the concatenation of myriad smaller subsequences each of which is locally homogeneous and of low localized entropy. The boundaries of these localized subsequences can be identified implicitly via adaptive transformations such as move-to-front, and need not be explicit; and the fact that they are of low-localized entropy means that a subsequent entropy coding step achieves high compression rates within each such subsequence.

Simply sorting the input sequence would lead to symbol-based runs of greatest length of course, minimizing the localized entropy. But that rearrangement would not be invertible. The second critical feature that makes the BWT indispensable is that it has an inverse, $\text{BWT}^{-1}$, that recovers the original sequence. These two features – and the fact that efficient implementations of both BWT and $\text{BWT}^{-1}$ have been developed – are why the BWT is widely used in lossless data compression applications such as `bzip2` [33].

The BWT is also a core component for powerful text indexing techniques [12, 13], some of which also support text search queries that run in $O(m)$ time, where $m$ is the length of the pattern. Such data structures are extensively used in the bioinformatics community for DNA sequencing and alignment tasks [19–22]. BWT is useful in text search applications due to its one-to-one correspondence with the *suffix array* [23]. A suffix array $SA$ for an $n$-sequence $S$ is an array of $n$ integer offsets (or pointers) into $S$, ordered such that the suffix strings indicated by those pointers are ordered lexicographically, that is, with $S[SA[i]\ldots] < S[SA[j]\ldots]$ whenever $i < j$. This simple data structure already supports efficient pattern search queries. For example, using standard binary search in the suffix array $SA$ allows patterns of length $m$ to be located in a sequence $S$ of length $n$ (with, typically, $m \ll n$) in $O(m \log n)$ time, once the suffix array has been constructed [23].

The one-to-one correspondence between the BWT and the suffix array also means that the BWT can be computed from a suffix array with a single linear scan. Moreover, suffix arrays

themselves can be constructed in linear time [16, 17, 30, 31]. In combination, these two results mean that the BWT for an $n$-sequence $S$ can be computed in $O(n)$ time, using a linear time suffix array construction algorithm as a central component.

The BWT is a critically important data transformation for both text compression and pattern search, which in turn means that it is highly desirable that the BWT's properties, including correctness, are formally verified. The work described in this paper, the formalization and functional correctness proof of the BWT and of its inverse transform $\text{BWT}^{-1}$, is the first of its kind, and provides the assurance required for operational confidence. The proof is mechanized in Isabelle/HOL [29].

## 2 Background

This section provides an overview of the BWT, its inverse transformation, and its correspondence with suffix arrays. It concludes with a brief overview of Isabelle/HOL. While this overview aims to be sufficiently self-contained for understanding our work, readers interested in a more detailed description of the BWT can refer to the work of Navarro [27, Chapter 11], further details about suffix arrays can be found in the work of Crochemore et al. [9, Chapter 4], and readers interested in Isabelle/HOL can refer to Nipkow and Klein's Isabelle/HOL textbook [28].

We assume throughout that sequences are over an ordered alphabet that has a special *minimum element* or *sentinel*, denoted as $, that occurs exactly once in any input sequence, particularly, in the last position. For example, in C strings the null byte that ends each string has the integer value zero. Sequences that satisfy this convention will be referred to as valid sequences.

### 2.1 The Burrows-Wheeler Transform

The BWT is a data transformation that produces an invertible permutation of a valid input sequence. The resulting permutation tends to be locally homogeneous, but globally heterogeneous, making it ideal for further processing by adaptive data compression techniques.

The canonical form of the BWT, denoted as bwt and first presented by Burrows and Wheeler [6], is shown in Algorithm 1. In overall terms, the construction can be thought of as starting with a valid $n$-sequence $S$; then generating all $n$ distinct rotations of $S$; then sorting the rotations lexicographically; and then extracting the last column of the reordered matrix of sequences. That final column also contains $n$ symbols and is the BWT-transformed sequence.

An example BWT computation is shown in Figure 1 for the sequence $S$ = yadayada$, with the BWT-transformed sequence $L$ = adyydaaa$ emerging on the right. Even at this very limited scale, the BWT tends to form groups of similar symbols. That tendency is amplified as the input sequence

**Algorithm 1** BWT Construction (general idea)

```
function bwt(S)
    M ← a matrix of all of the distinct rotations of S
    M′ ← sort the rows of M lexicographically
    L ← extract the last column of the sorted matrix
    return L
end function
```

| Rotations | | Sorted Rotations | | Last Column ($L$) |
|---|---|---|---|---|
| yadayada$ | | $yadayada | | a |
| adayada$y | | a$yadayad | | d |
| dayada$ya | | ada$yaday | | y |
| ayada$yad | ⇝ | adayada$y | ⇝ | y |
| yada$yada | | ayada$yad | | d |
| ada$yaday | | da$yadaya | | a |
| da$yadaya | | dayada$ya | | a |
| a$yadayad | | yada$yada | | a |
| $yadayada | | yadayada$ | | $ |

**Figure 1.** Constructing the BWT of the $n = 9$ symbols in the sequence $S$ = yadayada$. From left to right: the matrix $M$ of rotations of $S$; the sorted matrix $M'$; and the last column $L$ of $M'$, the BWT of $S$.

becomes longer. For example, when the LaTeX source code of a draft of this paper (approximately 50 kB) is run through the BWT, one part of the transformed sequence appears as limmmmmrrmmnml, and then just a hundred bytes away transitions to ssssssesshhess. Another hundred bytes further then gives pppbttlllpdpth.

To see why these clusters arise in BWT-transformed text, consider the character subsequence ainly. The suffix sorting process will bring every instance of that five-character pattern together, and the corresponding BWT characters – always one to the left of the corresponding suffix – must thus be derived from the words (assuming correctly spelled English input text across a typical literate vocabulary) ungainly, plainly, mainly, certainly, and vainly. That is, in the locus established by the suffix ainly the BWT sequence will be composed of the letters glmtv, with the proportions depending on the frequencies of those various words, and with the ordering of the letters determined by what comes after the ainly suffix, including the following spaces or punctuation characters. As an even more extreme example, in the sorted-suffix context established by the five-character sequence ghtly, the words rightly, nightly, tightly, brightly, lightly, and unsightly (plus similar words) mean that the BWT sequence will likely be a run containing nothing but i's. In the closely related (and thus nearby in the BWT sequence) suffix ghtle, the BWT symbols will also likely be dominated by i (because of weightless, flightless, and so on) but might also include sporadic instances of the letter u, from thoughtless.

| $F$ | | $L$ |
|---|---|---|
| $\$_8$ | $y_0a_1d_2a_3y_4a_5d_6$ | $a_7$ |
| $a_7$ | $\$_8y_0a_1d_2a_3y_4a_5$ | $d_6$ |
| $a_5$ | $d_6a_7\$_8y_0a_1d_2a_3$ | $y_4$ |
| $a_1$ | $d_2a_3y_4a_5d_6a_7\$_8$ | $y_0$ |
| $a_3$ | $y_4a_5d_6a_7\$_8y_0a_1$ | $d_2$ |
| $d_6$ | $a_7\$_8y_0a_1d_2a_3y_4$ | $a_5$ |
| $d_2$ | $a_3y_4a_5d_6a_7\$_8y_0$ | $a_1$ |
| $y_4$ | $a_5d_6a_7\$_8y_0a_1d_2$ | $a_3$ |
| $y_0$ | $a_1d_2a_3y_4a_5d_6a_7$ | $\$_8$ |

**Figure 2.** The sorted rotations matrix $M'$ of the sequence $S$ = yadayada\$ (see Figure 1) with each symbol labeled by its position in the sequence. The first and last columns are labeled as $F$ and $L$, respectively.

In data compression applications, further transformations, such as *move-to-front*, are applied to the BWT sequences, with each highly localized section of the BWT text giving rise to long runs of small integers. The final step applied to the stream of integers is then entropy coding [25].

Finally in this subsection, note that Figure 1 is somewhat misleading, since it suggests that the entire square matrix $M'$ is constructed, which would required $O(n^2)$ space and thus time. In fact it is not necessary to compute every element in $M'$ in order to determine $L$, a saving that will become apparent shortly.

### 2.2 The Inverse Transform

The fact that the BWT is invertible is a consequence of the following three sorted rotations matrix properties.

**Proposition 2.1** (Sorted Rotations Matrix Properties)**.** *The first column $F$ and last of column $L$ of the sorted rotations matrix $M'$ are connected in that:*

1. *The first column of the sorted rotations matrix, $F$, is obtained by sorting the BWT-transformed sequence, $L$. A simple counting sort is sufficient because the alphabet is known, and is (usually) small.*
2. *For all $i$, the symbol $L[i]$ occurs immediately to the left of the character $F[i]$ in the original sequence $S$, except in the case when $L[i]$ is the sentinel symbol \$.*
3. *The relative order for all equal-value symbols is the same in $F$ and in $L$.*

These three properties can be observed in Figure 2, which shows the sorted rotation matrix $M'$ for the input sequence $S$ = yadayada\$, where each symbol is annotated with its original position in $S$. For example, when $i = 1$ we have $F[i] = a_7$ and $L[i] = d_6$, in accordance with property (2). Similarly, and as asserted by property (3), consider the order in which the a symbols occur. In both $F$ and $L$ the order is $a_7$, $a_5$, $a_1$ and then $a_3$.

**Table 1.** The values arising from the rank function for the BWT-transformed sequence $L$ = adyydaaa\$.

| rank $L$ $v$ $i$ | | $i$ | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| $v$ | \$ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| | a | 0 | 1 | 1 | 1 | 1 | 1 | 2 | 3 | 4 | 4 |
| | d | 0 | 0 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 |
| | y | 0 | 0 | 0 | 1 | 2 | 2 | 2 | 2 | 2 | 2 |

**Table 2.** The values arising from the select function for the sorted sequence $F$ = \$aaaaddyy.

| select $F$ $v$ $i$ | | $i$ | | | |
|---|---|---|---|---|---|
| | | **0** | 1 | 2 | 3 |
| $v$ | \$ | **0** | - | - | - |
| | a | **1** | 2 | 3 | 4 |
| | d | **5** | 6 | - | - |
| | y | **7** | 8 | - | - |

To understand *why* property (3) must always hold, note that the ordering in each same-symbol grouping in column $F$ is determined by the suffixes that follow after that symbol, exactly because all of the rotations in that group *do* start with the same symbol. Moreover, when each of those "same initial symbol" rotations is shifted by one so that the column $F$ item goes to column $L$, the row ordering is again determined by the suffixes that follow those shifted symbols, with the first element of each of those suffixes now the dominant ordering basis in column $F$. That is, the tie-breaking rule for sorting the rotations that have the same first symbol is the same as the overall sorting rule for sorting the rotations. And hence all pairs of same-symbol items in column $F$ must appear in the same relative order in column $L$, and vice versa.

Using Proposition 2.1, the inverse of the BWT, $\text{BWT}^{-1}$, can be computed using the following two query functions.

- The rank function, which when given a sequence $xs$, a value $v$ and an index $i$, returns the number of times $v$ occurs in $xs$ up to but not including the $i^{\text{th}}$ position.
- The select function, which when given a sequence $xs$, a value $v$ and a natural number $i$, returns the position in $xs$ at which the $i^{\text{th}}$ occurrence of $v$ appears, where counting starts from 0.

To implement the $\text{BWT}^{-1}$ the rank function is used on the BWT-transformed sequence, and the select function is applied to the sorted sequence. Examples of these functions for the example sequence presented in Figure 1 are shown in Tables 1 and 2, respectively.

The rank function can be pre-computed in linear time, for constant sized alphabets. To do so, it suffices to iterate over the BWT-transformed sequence once for each symbol in the alphabet, tracking how frequently each symbol occurs.

**Algorithm 2** $\text{BWT}^{-1}$ Construction

```
function bwt^{-1}(L)
    S : the sequence being constructed
    F ← sort L
    n ← |L| − 1
    v ← $
    S[n] ← v
    i ← scan L to find where the sentinel symbol $ occurs
    repeat
        n ← n − 1
        j ← rank L v i
        k ← select F v 0
        i ← k + j
        v ← L[i]
        S[n] ← v
    until n = 0
    return S
end function
```

The select function can also be pre-computed in linear time for sorted sequences. This is because only the value for the $0^{\text{th}}$ occurrence is needed, as the $i^{\text{th}}$ occurrence (if it exists) can be obtained by adding $i$ to the position to the $0^{\text{th}}$ occurrence; that is, select $F\ v\ i$ = (select $F\ v\ 0$) + $i$. The $0^{\text{th}}$ occurrences can be obtained by computing the frequencies of all the symbols and summing them up accordingly using a process similar to counting sort.

Using pre-computed rank and select functions, and assuming that the linear array space required to store them is not an obstacle, the $\text{BWT}^{-1}$, denoted as $\text{bwt}^{-1}$, can be computed, regenerating the original sequence $S$ in right-to-left (that is, reverse) order, as shown in Algorithm 2. Broadly speaking, the inverse works by mapping symbols in $L$ to their corresponding locations in $F$, with this translation often referred to as the *LF-mapping*. The LF-mapping utilizes the fact that the $i^{\text{th}}$ occurrence of a symbol $v$ in $L$ corresponds to the $i^{\text{th}}$ occurrence of $v$ in $F$, drawing on property (3) of Proposition 2.1. Property (2) of Proposition 2.1 then allows identification of the predecessor of that symbol, thus allowing the array $S[]$ to be filled from right to left.

That is, the LF-mapping established by the calls to rank$L v i$ and select $F\ v\ 0$, in conjunction with property (3) of Proposition 2.1 ensures that paired symbols between $L$ and $F$ with the same value and the same rank for that value are correctly identified; and then property (2) of Proposition 2.1 allows the symbol that directly precedes the current one to be found. Overall, the original sequence is recovered in right-to-left order, following the chain of predecessor relationships between column $F$ and column $L$; an iteration that starts by locating the sentinel $ into $S[n]$ and ends once $S[0]$ has been reached as a predecessor and its value has been assigned.

For example, the first a symbol in both $L$ and $F$ is denoted $\text{a}_7$, as shown in Figure 2. This means that in the original sequence this particular a is located at position 7. Moreover, using property (2), the symbol that occurs before this can be obtained by looking up $L$ at the position where the first a symbol occurs in $F$. A complete computation of the $\text{BWT}^{-1}$ is shown for the $L$ = adyydaaa$ in Table 3.

**Table 3.** Inverting BWT-transformed sequence $L$ = adyydaaa$ to regenerate the original sequence $S$ = yadayada$, extracting symbols in order from $S[8]$ back to $S[0]$.

| $v$ | $i$ | $j$ = rank $L\ v\ i$ | $k$ = select $F\ v\ 0$ | $L[k+j]$ |
|---|---|---|---|---|
| - | - | - | - | $ |
| $ | 8 | 0 | 0 | a |
| a | 0 | 0 | 1 | d |
| d | 1 | 0 | 5 | a |
| a | 5 | 1 | 1 | y |
| y | 2 | 0 | 7 | a |
| a | 7 | 3 | 1 | d |
| d | 4 | 1 | 5 | a |
| a | 6 | 2 | 1 | y |

| $i$ | Sorted Rotations | | $SA[i]$ | Sorted Suffixes |
|---|---|---|---|---|
| 0 | $yadayada | | 8 | $ |
| 1 | a$yadayad | | 7 | a$ |
| 2 | ada$yaday | | 5 | ada$ |
| 3 | adayada$y | | 1 | adayada$ |
| 4 | ayada$yad | ⇄ | 3 | ayada$ |
| 5 | da$yadaya | | 6 | da$ |
| 6 | dayada$ya | | 2 | dayada$ |
| 7 | yada$yada | | 4 | yada$ |
| 8 | yadayada$ | | 0 | yadayada$ |

**Figure 3.** The correspondence between the sorted rotations matrix and the sorted suffix list for sequence $S$ = yadayada$. On the left of the sorted list of suffixes is the suffix array $SA$, which contains the suffix starting positions in sorted order.

### 2.3 One-to-One Correspondence with Suffix Arrays

As mentioned earlier, the BWT has a one-to-one correspondence with the suffix array, meaning that the BWT can be used to solve string matching problems, and can be computed in linear time. The correspondence between the two becomes clear when comparing the sorted rotations implicitly generated during the construction of the BWT and the sorted suffixes that are generated during the construction of the suffix array. Figure 3 shows this correspondence for the same example sequence $S$.

Each suffix is a prefix of exactly one rotation. Moreover, given that a suffix is represented by its starting location within the sequence, a rotation can similarly be represented by the number of left shifts that are needed to obtain it from the sequence. Hence, the mapping between rotations and

suffixes is such that the $i^{\text{th}}$ rotation maps to the $i^{\text{th}}$ suffix and vice versa. For example, the first rotation adayada$y maps to the first suffix adayada$ in Figure 3.

Importantly, the mapping between rotations and suffixes is monotonic and preserves the lexicographical ordering. This is because the comparison only ever compares up to the first occurring sentinel symbol $, as the sentinel is both the smallest symbol in the alphabet and also occurs only once in the input sequence, meaning that there will be no such case where two distinct rotations will have $ in the same location. Therefore, the comparison is equivalent to comparing the corresponding suffixes, and hence, lexicographical ordering is preserved. For example, the first rotation adayada$y is less than the third rotation ayada$yada, and likewise the first suffix adayada$ is less than the third suffix ayada$.

Due to the monotonic one-to-one mapping between rotations and suffixes, the sorted rotations matrix and the sorted suffixes list also have a one-to-one mapping. Hence, the suffix array has a one-to-one mapping with the sorted rotations matrix, as demonstrated by Figure 3. The BWT, which is the last column $L$ of the sorted rotations matrix $M'$, can thus be obtained from the suffix array by using the fact that the symbol at the $i^{\text{th}}$ position of $L$ occurs in $S$ immediately preceding the symbol at the $i^{\text{th}}$ position of $F$. And $F$ can be obtained by taking the first symbol of each suffix in the sorted suffix list.

Formally, this is expressed as follows:

**Definition 2.2** (BWT and Suffix Array Correspondence)**.** Given a sequence $S$ of length $n$ and its suffix array $SA$, the $i^{\text{th}}$ value in the sequence $L = \text{bwt } S$, that is, applying the BWT to $S$, is:

$$L[i] = \begin{cases} S[n-1] & SA[i] = 0\,, \\ S[SA[i]-1] & \text{otherwise.} \end{cases}$$

Note the special case that arises when the suffix array value is equal to 0, corresponding to the original input sequence $S$. In this case, the value of $L$ is the last symbol of the original sequence, which is always the sentinel.

As an example of the suffix array to BWT mapping, consider Figure 3, again applied to the sequence $S = \texttt{yadayada\$}$. With $SA[0]$ equal to the value 8, Definition 2.2 correctly anticipates that the $0^{\text{th}}$ element of the BWT of $S$ is $S[8-1] = \text{a}$. Meanwhile, $SA[8]$ is 0, and hence the last element of the BWT of $S$ must be $L[8] = S[8] = \$$.

Note that, as was also the case in connection with the description of the BWT, the figures shown here are illustrative rather than operational. In particular, a suffix array does not require the use of $O(n^2)$ space to store $n$ explicit suffix sequences (which is how Figure 3 might be interpreted); rather it is the array $SA$ that is stored, together with a single copy of the underpinning sequence $S$.

### 2.4 Isabelle/HOL

Isabelle/HOL [29] is an interactive theorem prover that allows classical extensional reasoning about statements expressed in higher-order logic. To guarantee correctness, it uses the LCF [15] approach. In this approach, its underlying system is a small inference kernel that contains a core set of axioms. It then uses the inference engine to check that all proof steps are derivable from the core axioms. All proofs are then guaranteed to be correct so long as the Isabelle/HOL inference kernel and core axioms are correct. Isabelle/HOL has a rich set of formalized theories and useful facilities for handling type classes and polymorphism. In particular, it supports the use of locales, which provide a method for modularly defining local scopes where constants along with assumptions can be defined. Theorems proven inside the context of a locale can use these constants and implicitly share the locale's assumptions. Furthermore, locales can be instantiated to various concrete instances if the assumptions can be discharged for these particular instances.

## 3 BWT Formalization and Verification

HOL implementations of the BWT, shown in Algorithm 1, and of its inverse, shown in Algorithm 2, have been formalized in Isabelle/HOL. This builds on a recent formalization of suffix arrays [8]. In this work, additional properties about the correspondence between the BWT and suffix array were also formalized. Moreover, this work also contains a formalization of the rank and select functions, which are used in the inverse. The total size of the formalization is 3,844 lines of Isabelle/HOL. Out of this total, 1,787 lines are for the formalization of rank, select, and additional counting properties, 596 lines are for general properties on suffix arrays and bounded incrementing sequences, and the remaining 1,461 lines for the formalization of the BWT and its inverse.

### 3.1 Preliminaries

Before presenting the formalization of the BWT and its inverse, it is important to recall that the BWT requires sequences to have their elements drawn from an ordered alphabet that has a minimum symbol. In the formalization, these sequences are represented using Isabelle/HOL's polymorphic lists, whose elements are of any type that has a total order and a minimum element. In Isabelle/HOL this is natively expressed by stating that the element type is of sort `linorder` and `order_bot`, respectively.

Moreover, the formalization also utilizes two key constructs. These are the **valid** predicate and the suffix array construction function sa. The **valid** predicate is used to ensure that lists are terminated by the minimum element in the alphabet and that this minimum element does not occur anywhere else in the list. This is defined as follows.

**Definition 3.1** (Valid)**.** A list $S$, whose elements belong to an alphabet that has a total order and contains a minimum

element, is valid if it satisfies the following:

$$\textbf{valid}\, S \iff$$
$$|S| > 0 \wedge \text{last}\, S = \bot \wedge (\forall i.\, i < |S| - 1 \implies S[i] \neq \bot)\,,$$

where $\bot$ is the minimum and last returns the last element of a list.

The suffix array construction function sa, as the name suggests, constructs the suffix array of the list it is provided. Its characterization is defined as follows:

**Definition 3.2** (Suffix Array Construction Characterization)**.** A function sa is a suffix array construction function if and only if for all lists $S$, whose elements belong to a total order, the following is true:

$$(\text{sa}\, S) \sim [0..<|S|] \;\wedge \textbf{sorted}_{(<)}\, (\text{map}\, (\text{suffix}\, S\,)\, (\text{sa}\, S))$$

where $x \sim y$ means that $x$ is a permutation of $y$, $[i..<j]$ is the list from $i$ up to but not including $j$, and $\textbf{sorted}_{(<)}\, x$ means that $x$ is sorted w.r.t. the strictly less than comparison operator ($<$).

In the formalization, we do not utilize a specific suffix array construction function but rather use an Isabelle/HOL locale, called *Suffix-Array-General*, where sa is the parameter to the locale and Definition 3.2 is the locale assumption. Then the formalization of the BWT occurs within the locale context, and thus, has access to definitions and theorems within the locale.

Definition 3.1 and the *Suffix-Array-General* locale were developed in previous work [8] that showed that this locale uniquely characterizes suffix arrays. Moreover, two suffix array construction algorithms were formally verified as part of that work: one that uses a general sorting algorithm to sort suffixes, and a linear time algorithm called SA-IS [30]. While our work benefits from these results as they validate this characterization of a suffix array construction, our formalization only directly relies on the valid sequence predicate and locale itself, and hence, only requires one to understand Definitions 3.1 and 3.2.

### 3.2 Formalizing the BWT

In Sections 2.1 and 2.3, two different formulations of the BWT were presented, one using the canonical description, that would not be efficient to directly implement, and one using a suffix array formulation, which results in an efficient implementation. These are both formalized and shown to be equivalent.

The canonical form of the BWT, shown below, is essentially a transliteration of the pseudocode in Algorithm 1:

**Definition 3.3** (Canonical BWT)**.** For all $S$, such that **valid** $S$ holds, the canonical BWT definition is:

$$\text{bwt}\, S =$$
$$\text{map last}\, (\text{sort}\, (\text{map}\, (\lambda i.\, \text{rotate}\, i\, S)\, [0..<|S|]))\,.$$

It uses the in-built functions rotate to generate rotations, sort for sorting, and last to extract the last elements from the sorted list of rotations. A list $S$ of length $n$ has $n$ rotations, and it is thus sufficient to generate rotations for the values 0 to $n-1$.

The definition of the BWT in terms of the suffix array is similar to Definition 2.2. But instead of a piecewise function, uses the modulo (mod) operator for natural numbers:

**Definition 3.4** (Suffix Array Version of the BWT)**.** For all lists $S$ such that **valid** $S$, and suffix array construction function sa satisfying Definition 3.2, the BWT is computed as follows:

$$\text{bwt-sa}\, S = \text{map}\, (\lambda i.\, S[(i + |S| - 1) \bmod |S|])\, (\text{sa}\, S)\,.$$

Since indices are represented as natural numbers, the length of the list needs to be added when doing the modulo calculation. This is because, in the calculation above, one is subtracted from the index, which could potentially lead to an invalid result if not handled correctly. Adding the length of the sequence prevents this case from happening, and at the same time preserves the result of the modulo calculation.

To show that Definitions 3.3 and 3.4 are equivalent, the rotations and suffixes are first shown to have the same lexicographical ordering.

**Theorem 3.5** (Same Suffix and Rotation Order)**.** *For a list $S$, if* **valid** *$S$ holds and $i$, $j$ are less than $|S|$, then*

$$\text{suffix}\, S\, i < \text{suffix}\, S\, j \iff \text{rotate}\, i\, S < \text{rotate}\, j\, S\,.$$

The proof of this is straightforward and reduces to showing that:

1. the $i^{\text{th}}$ suffix is a prefix of the $i^{\text{th}}$ rotation and
2. the lexicographical comparison between rotations only ever compares up to the first (and only) occurrence of the minimum element \$.

Both of these follow from the definition of a rotation and of lexicographical order.

Theorem 3.5 implies that generating the rotations from the suffix array is equivalent to generating and then sorting the rotations. With this and the fact that taking the last element of the $i^{\text{th}}$ rotation of a sequence $S$ is equal to

$$S[(i + |S| - 1) \bmod |S|]$$

the equivalence of the different formulations of the BWT is obtained; as stated by the following theorem.

**Theorem 3.6** (BWT and Suffix Array Correspondence)**.** *For a list $S$,*

$$\textbf{valid}\, S \implies \text{bwt}\, S = \text{bwt-sa}\, S\,.$$

### 3.3 Rank and Select

Before moving to the inverse transform, we briefly describe the formalization of the rank and select functions and some key properties that are necessary to understand the rest of the formalization. The purpose of this formalization is to

define the semantics and properties of rank and select. Efficient implementations are outside the scope of this work, but we note that numerous implementations have been developed and formally verifying such efficient implementations deserves its own separate investigation. The definitions for rank and select, as well as some of their properties, are in part based on the work of Affeldt [1]. However, we also formalize several additional properties that turned out to be necessary for our formalization efforts, such as Theorems 3.10 and 3.11.

Recall that rank $xs\ v\ i$ returns the number of occurrences of the value $v$ in the list $xs$ up to (but not including) the $i^{\text{th}}$ position. This can be computed using Isabelle/HOL's count-list function, which counts how many times a value occurs in a list, and the take function, which returns the prefix up to the $i^{\text{th}}$ location:

**Definition 3.7** (Rank)**.**

$$\mathsf{rank}\ xs\ v\ i = \mathsf{count\text{-}list}\ (\mathsf{take}\ i\ xs)\ v\ .$$

The select $xs\ v\ i$ function returns the location of the $i^{\text{th}}$ occurrence of a value $v$ in the list $xs$. We define this using a recursive function that iterates over the list, decrementing $i$ each time the current element equals $v$. When $i$ is equal to zero and the current value is equal to $v$, that location is returned. If such a scenario does not occur, then the return value is $|xs|$:

**Definition 3.8** (Select)**.**

$$\begin{aligned}
&\mathsf{select}\ [\,]\ v\ i = 0\ |\\
&\mathsf{select}\ (a\#xs)\ v\ 0 =\\
&\quad(\mathsf{if}\ v = a\ \mathsf{then}\ 0\ \mathsf{else}\ \mathsf{Suc}\ (\mathsf{select}\ xs\ v\ 0))\ |\\
&\mathsf{select}\ (a\#xs)\ v\ (\mathsf{Suc}\ i) =\\
&\quad(\mathsf{if}\ v = a\ \mathsf{then}\ \mathsf{Suc}\ (\mathsf{select}\ xs\ v\ i)\\
&\quad\ \mathsf{else}\ \mathsf{Suc}\ (\mathsf{select}\ xs\ v\ (\mathsf{Suc}\ i)))\ .
\end{aligned}$$

The select function is slightly more complex than rank, and required a correctness theorem.

**Theorem 3.9** (Correctness of Select)**.**

$$\begin{aligned}
&\mathsf{select}\ xs\ v\ i \le |xs| \wedge\\
&(\mathsf{select}\ xs\ v\ i < |xs| \implies \mathsf{rank}\ xs\ v\ (\mathsf{select}\ xs\ v\ i) = i) \wedge\\
&(\mathsf{select}\ xs\ v\ i = |xs| \implies \mathsf{count\text{-}list}\ xs\ v \le i)\ .
\end{aligned}$$

Essentially, this guarantees that if select returns a valid index, then it corresponds to the $i^{\text{th}}$ occurrence of $v$ in $xs$. Otherwise, it must mean that there is no $i^{\text{th}}$ occurrence $v$ in $xs$. The above is proved by induction on the list.

The description of the $\text{BWT}^{-1}$ provided earlier stated that if the list is sorted, then only select $xs\ v\ 0$ needs to be known. We prove this by induction on the list:

**Theorem 3.10** (Select Sorted Equivalence)**.** *If $xs$ is sorted and $i <$ count-list $xs\ v$ then*

$$\mathsf{select}\ xs\ v\ i = \|\{k.\ k < |xs| \wedge xs[k] < v\}\| + i\ ,$$

*where $\|s\|$ returns the cardinality of a given set $s$.*

An alternative formulation for rank is now possible, capturing the synergy with Theorem 3.10:

**Theorem 3.11** (Rank Equivalence)**.**

$$\mathsf{rank}\ xs\ v\ i = \|\{k.\ k < |xs| \wedge k < i \wedge xs[k] = v\}\|\ .$$

Proving this theorem reduces to showing that

$$\mathsf{count\text{-}list}\ xs\ v = \|\{k.\ k < |xs| \wedge xs[k] = v\}\|\ .$$

This is proven by straightforward induction on the list. Then the rest of the proof of Theorem 3.11 follows from this fact along with Definition 3.7.

### 3.4 Sorted Rotations Matrix Properties

The LF-mapping is a core component of $\text{BWT}^{-1}$. Recall that this maps values in the BWT-transformed sequence $L$, (the last column of the sorted rotations matrix $M'$) to the values in the sorted sequence $F$, the first column in the sorted rotations matrix. The mapping demonstrates that the $i^{\text{th}}$ occurrence of $v$ in $L$ is also the $i^{\text{th}}$ occurrence of $v$ in $F$.

The LF-mapping is a consequence of Proposition 2.1, and so, these properties need to be shown to be true. This is achieved by using the correspondence with suffix arrays. Specifically, the BWT permutation specifies how the indices of the original sequence are permuted to produce the BWT, which is defined in terms of the suffix array as follows:

**Definition 3.12** (BWT Permutation)**.** For a list $S$, where **valid** $S$ holds, and suffix array construction function sa, the BWT permutation is:

$$\mathsf{bwt\text{-}perm}\ S = \mathsf{map}\ (\lambda i.\ (i + |S| - 1)\ \mathsf{mod}\ |S|)\ (\mathsf{sa}\ S)\ .$$

Property (2) of Proposition 2.1 immediately follows. In addition, property (1) is also trivial to prove, as the suffix array can be viewed as the permutation mapping of $F$, the first column of the sorted matrix. Hence for valid list $S$,

$$F = \mathsf{map}\ (\lambda i.\ S[i])\ (\mathsf{sa}\ S)\ ,$$

which can easily be shown to be sorted as a consequence of the suffix array representing the sorted suffixes $S$, as per Definition 3.2.

Property (3) of Proposition 2.1, shown below, is also straightforward to prove, and again follows from property (2) of Proposition 2.1, Definition 3.2, and the definition of lexicographical order.

**Theorem 3.13** (Suffix Relative Order Preservation)**.** *Given a valid list $S$, if $i < j$, $j < |S|$ then*

$$\begin{aligned}
&S[(\mathsf{bwt\text{-}perm}\ S)[i]] = S[(\mathsf{bwt\text{-}perm}\ S)[j]] \implies\\
&\mathsf{suffix}\ S\ ((\mathsf{bwt\text{-}perm}\ S)[i]) < \mathsf{suffix}\ S\ ((\mathsf{bwt\text{-}perm}\ S)[j])\ .
\end{aligned}$$

Note that in the formalization, only properties (1) and (3) are explicitly proven, as property (2) is immediately obtained by term rewriting.

### 3.5 Verifying the Correctness of the Inverse

The Isabelle/HOL embedding of the $\text{BWT}^{-1}$, $\text{bwt}^{-1}$, closely resembles Algorithm 2 presented in Section 2. It uses a loop, in this case a recursive function $\text{bwt}_\text{n}^{-1}$, that repeatedly applies the LF-mapping, lf-map, to find the next symbol in the original sequence in reverse order. These are defined as follows:

**Definition 3.14** ($\text{BWT}^{-1}$).

$$\text{lf-map}\ ss\ bs\ i = (\text{select}\ ss\ bs[i]\ 0) + (\text{rank}\ bs\ bs[i]\ i)$$

---

$$\begin{aligned}&\text{bwt}_\text{n}^{-1}\ 0\ ss\ bs\ i = [\,]\ |\\&\text{bwt}_\text{n}^{-1}\ (\text{Suc}\ n)\ ss\ bs\ i =\\&\quad(\text{bwt}_\text{n}^{-1}\ n\ ss\ bs\ (\text{lf-map}\ ss\ bs\ i))\ @\ [bs[i]]\end{aligned}$$

---

$$\text{bwt}^{-1}\ bs = \text{bwt}_\text{n}^{-1}\ |bs|\ (\text{sort}\ bs)\ bs\ (\text{select}\ bs\ \bot\ 0)\,,$$

where @ is the concatenation operator for two lists.

For $\text{bwt}^{-1}$ to be correct, it needs to satisfy:

$$\textbf{valid}\ S \implies \text{bwt}^{-1}\ (\text{bwt}\ S) = S\,,$$

This is achieved by verifying the correctness of lf-map and $\text{bwt}_\text{n}^{-1}$. These in turn are proven correct by proving the correctness of $\text{lf-map}_\alpha$, an abstract version of lf-map, and $\text{bwt-perm}_\alpha^{-1}$, an abstract version of the $\text{BWT}^{-1}$ permutation, respectively, and then verifying that these abstract versions are equivalent to the concrete ones. We call $\text{lf-map}_\alpha$ and $\text{bwt-perm}_\alpha^{-1}$ abstract as these have access to additional information about the original sequence and its suffix array. When these are made concrete, all information about the original sequence and its suffix array are removed, and only information that can be derived from the BWT-transformed sequence itself remains.

The abstract LF-mapping, $\text{lf-map}_\alpha$, defined below, takes the original list $S$, and a position $i$ in $L$, the last column of the sorted rotations matrix and BWT-transformed sequence of $S$. It returns the location in $F$ (first column of the sorted rotations matrix) of the value $L[i]$. That is, given the $k^{\text{th}}$ occurrence of some value $v$ in $L$, it returns the $k^{\text{th}}$ occurrence of $v$ in $F$.

**Definition 3.15** (Abstract LF-mapping). Given a list $S$, such that **valid** $S$, then

$$\begin{aligned}&\text{lf-map}_\alpha\ S\ i =\\&\quad\text{select}\ (\text{sort}\ S)\ (\text{bwt-sa}\ S)[i]\ (\text{rank}\ (\text{bwt-sa}\ S)\ S[i]\ i)\,.\end{aligned}$$

The abstract $\text{BWT}^{-1}$ permutation, defined below, is essentially the repeated application the abstract LF-mapping. Note that this is referred to as abstract, as it too uses the original $S$ rather than the BWT-transformed sequence.

**Definition 3.16** ($\text{BWT}^{-1}$ Permutation).

$$\begin{aligned}&\text{bwt-perm}_\alpha^{-1}\ 0\ S\ i = [\,]\ |\\&\text{bwt-perm}_\alpha^{-1}\ (\text{Suc}\ n)\ S\ i =\\&\quad\text{bwt-perm}_\alpha^{-1}\ n\ S\ (\text{lf-map}_\alpha\ S\ i)\ @\ [i]\,.\end{aligned}$$

Strictly speaking, the above returns a sub-list of a rotation of the $\text{BWT}^{-1}$ permutation, as it depends on the initial value of $i$ and the number of times $\text{lf-map}_\alpha$ is applied. If the initial starting value is the location of the minimum value $\bot$ in the BWT-transformed sequence, and it iterates for the length of the sequence, then it will return the $\text{BWT}^{-1}$ permutation.

The refinement proofs from the abstract definitions to the concrete ones, shown in Definition 3.14, are omitted from the presentation as they are straightforward and are largely proved using standard term rewriting. For example, the refinement from $\text{lf-map}_\alpha$ to lf-map follows from the fact that for a sequence $S$,

$$\text{sort}\ S = \text{sort}\ (\text{bwt-sa}\ S)\,,$$

and from Theorem 3.10,

$$\text{select}\ (\text{sort}\ S)\ v\ i = \text{select}\ (\text{sort}\ S)\ v\ 0 + i\,.$$

A similar argument is also used for the refinement from the abstract to the concrete versions of the $\text{BWT}^{-1}$ permutation.

To show that Definition 3.15 is correct, the $k^{\text{th}}$ occurrences of $v$ in $L$ and $F$ need to be shown to have come from the same location in $S$. This can be shown using Theorem 3.13 and is formalized as follows:

**Theorem 3.17** (Same Rank). *For a valid list $S$, if $i$ and $j$ are less than $|S|$, and $S[(\text{sa}\ S)[i]]$ and $S[(\text{bwt-perm}\ S)[j]]$ are equal to $v$ then*

$$\begin{aligned}&\text{rank sort}\ S\ v\ i = \text{rank bwt-sa}\ S\ v\ j \implies\\&(\text{sa}\ S)[i] = (\text{bwt-perm}\ S)[j]\,.\end{aligned}$$

The correctness of the abstract LF-mapping then follows:

**Theorem 3.18** (Abstract LF-Mapping Correctness). *For a valid list $S$ and $i < |S|$, then*

$$\begin{aligned}&(\text{bwt-perm}\ S)[\text{lf-map}_\alpha\ S\ i] =\\&\quad((\text{bwt-perm}\ S)[i] + |S| - 1)\bmod |S|\,.\end{aligned}$$

Theorem 3.18 states that $\text{lf-map}_\alpha$ finds the next value in the $\text{BWT}^{-1}$ permutation in reverse. Hence, repeated applications of $\text{lf-map}_\alpha$ by $\text{bwt-perm}_\alpha^{-1}$ should produce the whole $\text{BWT}^{-1}$ permutation. One slight complication is that depending on what initial starting index is used, the $\text{BWT}^{-1}$ permutation may be rotated.

To solve this, the following is useful:

**Theorem 3.19** (Abstract $\text{BWT}^{-1}$ Permutation is a Rotated Sub-list). *Suppose that $S$ is a list such that* **valid** *$S$, $i < |S|$, and $n \le |S|$, then*

$$\exists k\ ys\ zs.\ \text{rotate}\ k\ [0..<|S|] = ys\ @\ xs\ @\ zs\,,$$

*where*

$$xs = \mathsf{map}\ (\lambda j.\ (\mathsf{bwt\text{-}perm}\ S)[j])\ (\mathsf{bwt\text{-}perm}_\alpha^{-1}\ n\ S\ i)\ .$$

This follows from the fact that repeated applications of $\mathsf{lf\text{-}map}_\alpha$ produce a sequence that when passed to bwt-perm, becomes a bounded incrementing sequence, which is a sequence that increases by one, eventually wrapping back to zero after reaching the maximum $|S| - 1$. While there are several additional proof steps involved, the core component follows from Theorem 3.18.

From Definition 3.16, it becomes clear that a list of length $n$ will be produced when the number of iterations is $n$, with $n$ being the first argument. Moreover, if the initial value of $i$ is equal to the location of $\perp$ in the BWT-transformed sequence $L$, that is, $i = \mathsf{select}\ L \perp 0$, the following holds:

$$(\mathsf{bwt\text{-}perm}\ S)[(\mathsf{bwt\text{-}perm}_\alpha^{-1}\ |S|\ S\ i)[|S| - 1]] = |S| - 1\ .$$

Combining that with Theorems 3.6 and 3.19 and the refinement theorems, the $\mathrm{BWT}^{-1}$ correctness can then be obtained.

**Theorem 3.20** (Correctness of the $\mathrm{BWT}^{-1}$)**.** *For a valid list $S$,*

$$\mathsf{bwt}^{-1}\ (\mathsf{bwt}\ S) = S\ .$$

# 4 Discussion

In this section, we present our reasoning for formalizing the BWT using suffix arrays, note some difficulties that were encountered, and consider how to instantiate Definitions 3.4 and 3.14 into efficient implementations.

## 4.1 The BWT in Terms of Suffix Arrays

In our formalization of the BWT and its inverse, we utilized the one-to-one correspondence between the BWT and suffix array. While it is possible to specify the BWT without using suffix arrays, we found that doing so was more complicated. This is because suffix arrays succinctly express positional information, notably as captured by Definition 3.2, which is useful for the verification of the inverse. For example, the BWT permutation, Definition 3.12, is expressed in terms of the suffix array.

Another benefit of using suffix arrays is that they are also useful in a computational sense, allowing the BWT for a sequence to be calculated in a simple manner. Furthermore, if we have a formally verified suffix array construction algorithm, we can easily plug that into our specification to obtain a formally verified BWT construction algorithm. An algorithm for computing the BWT will then inherit the same time complexity as the suffix array construction algorithm, since any suffix array construction algorithm must require time that is $O(n)$ or greater.

## 4.2 Formalization Challenges

One of the major difficulties encountered was capturing the logic of why the BWT is invertible. While there exist many explanations for this, the majority of these borrow heavily on intuition rather than logic. While such explanations are suitable for many needs, it is the precise details that are necessary in works like ours. Other descriptions, ones that sought to provide detailed and exact explanations, were difficult to follow.

In terms of theorem proving, while no major obstacles were preventing the completion of the proofs, there was little existing theory on properties about suffix arrays, finding the $k^{\text{th}}$ occurrence of some value in a list and bounded incrementing sequences. Hence, many of these were formalized in this work, becoming a significant proportion of the formalization. That is, over half the total of the lines of Isabelle/HOL were for formalizing these properties. Theorems that were the intersection of two or more of these areas were particularly difficult to prove, such as Theorems 3.17 and 3.19. These can hopefully be reused by others.

## 4.3 Limitations and Future Work

Our formalization of the BWT and its inverse is not intended to produce efficient implementations. Rather, it is meant to be a mathematical formalization, specifying logical properties about the BWT. However, the embedding of BWT that is computed via a suffix array, that is, Definition 3.4, can be made into a linear time implementation, given a linear time suffix array construction function. While such an implementation would be fast, it does have a larger memory overhead, as the whole suffix array needs to be constructed and stored before the BWT sequence can be generated. This might not be suitable for very large inputs.

The inverse transformation, $\mathrm{BWT}^{-1}$, presented in Definition 3.14, requires further refinement into a linear time implementation. At present, the rank and select functions are implemented rather naïvely. However, this could be resolved by providing an efficient pre-computation and verifying that it is a refinement of our straightforward implementation.

The select function could be replaced with a data structure such as that shown in Table 2, that can be computed by an approach similar to counting sort. The partner rank function could be computed by calculating the cumulative frequencies for each symbol in the text. For constant-sized alphabets, this would still provide a linear time computation. However, this approach has a linear-space memory overhead. Reductions are possible but would affect the running time, and hence change the overall running time of the inverse BWT.

# 5 Related Work

There is an extensive literature concerning the BWT in the areas of information theory [24], combinatorial analysis [14, 32], BWT variants [10, 18], and many more; but very little in terms of formalizing it. The closest work in the same vein as ours is by Bird et al. [3] on constructing the inverse BWT in Haskell using equational reasoning to generate the

sorted rotations matrix. However, it is largely incomparable, as ours utilizes the correspondence with suffix arrays and is a mechanized formalization of an abstract embedding.

Formalizations of suffix array construction algorithms are closely connected, as the BWT and the suffix array have a one-to-one correspondence. In this area there have been a number of formally verified suffix array construction algorithms [4, 5, 11]. However, all of these are in connection with simple but inefficient algorithms that use general string sorting algorithms to order the suffixes, rather than the more efficient bespoke linear time algorithms. An exception to this is the SA-IS algorithm [30], a linear time suffix array construction algorithm, that was verified in Isabelle/HOL in previous work [8]. However, while this can be plugged into our formalization of the BWT, resulting in a linear time BWT construction algorithm, it still would not lead to an efficient implementation as the SA-IS algorithm formalization is still just a HOL implementation, albeit extractable to Haskell.

In a similar vein, formalizations of rank and select data structures are also of interest, as these are required in order to compute the BWT inverse. Moreover, algorithms that search for patterns in BWT-transformed text often utilize such data structures [13]. In this area, there exist several formalizations [1, 34] in Coq [2]. It would be interesting to compose their work with ours; however, since our work is carried out in Isabelle/HOL, this might also be challenging.

## 6 Conclusion and Future Work

We have presented the first formal verification of both the BWT and its inverse. The formalization consists of a mechanized proof of correctness, invertibility and termination for both transformations, using Isabelle/HOL. We have thus addressed an important gap in the formal verification of compression algorithms, ensuring that the BWT can be safely deployed in critical applications. In particular, this work provides the necessary foundation that is required for verifying the various algorithms for compression and text search that operate on BWT-transformed sequences.

While we have addressed the mathematical description of the BWT, the practical side still needs further attention, including formally verifying time- and space-efficient implementations that construct the BWT and its inverse. Potential avenues for future work thus include verifying efficient implementations of linear time suffix array construction algorithms [16, 17, 30, 31], and efficient structures for computing rank and select queries [26]. Besides the construction of the BWT and its inverse, algorithms and data structures utilizing the BWT are also potential targets [12, 13].

## 7 Data-Availability Statement

The Isabelle/HOL formalization of the BWT and its inverse is available online [7].

## Acknowledgments

This research is partially supported by an Australian Government Research Training Program (RTP) Scholarship.

## References

[1] R. Affeldt, J. Garrigue, X. Qi, and K. Tanaka. 2019. Proving Tree Algorithms for Succinct Data Structures. In *Proc. Interactive Theorem Proving (LIPIcs, Vol. 141)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 5:1–5:19. https://doi.org/10.4230/LIPICS.ITP.2019.5

[2] Y. Bertot and P. Castéran. 2004. *Interactive Theorem Proving and Program Development - Coq'Art: The Calculus of Inductive Constructions*. Springer. https://doi.org/10.1007/978-3-662-07964-5

[3] R. S. Bird and S. Mu. 2004. Inverting the Burrows-Wheeler Transform. *Journal of Functional Programming* 14, 6 (2004), 603–612. https://doi.org/10.1017/S0956796804005118

[4] F. Bobot, J. Filliâtre, C. Marché, and A. Paskevich. 2015. Let's Verify This with Why3. *International Journal on Software Tools for Technology Transfer* 17, 6 (2015), 709–727. https://doi.org/10.1007/s10009-014-0314-5

[5] D. Bruns, W. Mostowski, and M. Ulbrich. 2015. Implementation-Level Verification of Algorithms with KeY. *International Journal on Software Tools for Technology Transfer* 17, 6 (2015), 729–744. https://doi.org/10.1007/s10009-013-0293-y

[6] M. Burrows and D. Wheeler. 1994. *A Block-Sorting Lossless Data Compression Algorithm*. Technical Report. Digital SRC Research Report.

[7] L. Cheung and C. Rizkallah. 2024. Formalized Burrows-Wheeler Transform (Artefact). https://doi.org/10.5281/zenodo.14279882

[8] L. Cheung and C. Rizkallah. 2024. Formally Verified Suffix Array Construction. *Archive of Formal Proofs* (September 2024). https://isa-afp.org/entries/SuffixArray.html, Formal proof development.

[9] M. Crochemore, C. Hancart, and T. Lecroq. 2007. *Algorithms on Strings*. Cambridge University Press, Cambridge.

[10] J. W. Daykin and W. F. Smyth. 2014. A Bijective Variant of the Burrows-Wheeler Transform using V-order. *Theory of Computer Science* 531 (2014), 77–89. https://doi.org/10.1016/J.TCS.2014.03.014

[11] G. Ernst, J. Pfähler, G. Schellhorn, D. Haneberg, and W. Reif. 2015. KIV: Overview and VerifyThis Competition. *International Journal on Software Tools for Technology Transfer* 17, 6 (2015), 677–694. https://doi.org/10.1007/s10009-014-0308-3

[12] P. Ferragina and G. Manzini. 2000. Opportunistic Data Structures with Applications. In *Foundations of Computer Science*. IEEE Computer Society, 390–398. https://doi.org/10.1109/SFCS.2000.892127

[13] P. Ferragina and G. Manzini. 2005. Indexing Compressed Text. *Journal of ACM* 52, 4 (2005), 552–581. https://doi.org/10.1145/1082036.1082039

[14] R. Giancarlo, A. Restivo, and M. Sciortino. 2007. From First Principles to the Burrows and Wheeler Transform and Beyond, via Combinatorial Optimization. *Theory of Computer Science* 387, 3 (2007), 236–248. https://doi.org/10.1016/J.TCS.2007.07.019

[15] M. Gordon, R. Milner, and C. P. Wadsworth. 1979. *Edinburgh LCF: A Mechanised Logic of Computation*. Lecture Notes in Computer Science, Vol. 78. Springer.

[16] J. Kärkkäinen and P. Sanders. 2003. Simple Linear Work Suffix Array Construction. In *Proc. Automata, Languages and Programming (Lecture Notes in Computer Science, Vol. 2719)*. Springer, 943–955. https://doi.org/10.1007/3-540-45061-0_73

[17] P. Ko and S. Aluru. 2005. Space Efficient Linear Time Construction of Suffix Arrays. *Journal of Discrete Algorithms* 3, 2-4 (2005), 143–156. https://doi.org/10.1016/j.jda.2004.08.002

[18] M. Kufleitner. 2009. On Bijective Variants of the Burrows-Wheeler Transform. In *Proc. Prague Stringtology*. Prague Stringology Club, Department of Computer Science and Engineering, Faculty of Electrical Engineering, Czech Technical University in Prague, 65–79. http://www.stringology.org/event/2009/p07.html

[19] B. Langmead and S. L. Salzberg. 2012. Fast Gapped-Read Alignment with Bowtie 2. *Nature Methods* 9, 4 (01 Apr 2012), 357–359. https://doi.org/10.1038/nmeth.1923

[20] B. Langmead, C. Trapnell, M. Pop, and S. L. Salzberg. 2009. Ultrafast and Memory-efficient Alignment of Short DNA Sequences to the Human Genome. *Genome Biology* 10, 3 (04 Mar 2009), 25. https://doi.org/10.1186/gb-2009-10-3-r25

[21] H. Li and R. Durbin. 2009. Fast and Accurate Short Read Alignment with Burrows-Wheeler Transform. *Bioinformatics* 25, 14 (2009), 1754–1760. https://doi.org/10.1093/bioinformatics/btp324

[22] R. Li, C. Yu, Y. Li, T. W. Lam, S. Yiu, K. Kristiansen, and J. Wang. 2009. SOAP2: An Improved Ultrafast Tool for Short Read Alignment. *Bioinformatics* 25, 15 (2009), 1966–1967. https://doi.org/10.1093/bioinformatics/btp336

[23] U. Manber and E. W. Myers. 1993. Suffix Arrays: A New Method for On-Line String Searches. *SIAM J. Comput.* 22, 5 (1993), 935–948. https://doi.org/10.1137/0222058

[24] G. Manzini. 2001. An Analysis of the Burrows-Wheeler Transform. *Journal of ACM* 48, 3 (2001), 407–430. https://doi.org/10.1145/382780.382782

[25] A. Moffat and A. Turpin. 2002. *Compression and Coding Algorithms*. Kluwer Academic Publishers, Boston, MA.

[26] G. Navarro. 2014. Wavelet Trees For All. *Journal of Discrete Algorithms* 25 (2014), 2–20. https://doi.org/10.1016/J.JDA.2013.07.004

[27] G. Navarro. 2016. *Compact Data Structures - A Practical Approach*. Cambridge University Press, Cambridge.

[28] T. Nipkow and G. Klein. 2014. *Concrete Semantics - With Isabelle/HOL*. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-319-10542-0

[29] T. Nipkow, L. C. Paulson, and M. Wenzel. 2002. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science, Vol. 2283. Springer. https://doi.org/10.1007/3-540-45949-9

[30] G. Nong, S. Zhang, and W. H. Chan. 2009. Linear Suffix Array Construction by Almost Pure Induced-Sorting. In *Proc. Data Compression Conference*. IEEE Computer Society, 193–202. https://doi.org/10.1109/DCC.2009.42

[31] G. Nong, S. Zhang, and W. H. Chan. 2011. Two Efficient Algorithms for Linear Time Suffix Array Construction. *IEEE Trans. Computers* 60, 10 (2011), 1471–1484. https://doi.org/10.1109/TC.2010.188

[32] G. Rosone and M. Sciortino. 2013. The Burrows-Wheeler Transform between Data Compression and Combinatorics on Words. In *Proc. Nature of Computation. Logic, Algorithms, Applications (Lecture Notes in Computer Science, Vol. 7921)*. Springer, 353–364. https://doi.org/10.1007/978-3-642-39053-1_42

[33] J. Seward. 1996. bzip2 Homepage. https://sourceware.org/bzip2/index.html. Accessed: 2023-09-12.

[34] A. Tanaka, R. Affeldt, and J. Garrigue. 2016. Formal Verification of the Rank Algorithm for Succinct Data Structures. In *Proc. International Conference of Formal Engineering Methods (Lecture Notes in Computer Science, Vol. 10009)*. Springer, 243–260. https://doi.org/10.1007/978-3-319-47846-3_16